ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 19

Работа с реляционными СУБД на примере PostgreSQL

Рассмотрим процесс работы с реляционными системами управления базами данных на примере СУБД PostgreSQL. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Введение в реляционные базы данных и PostgreSQL

Реляционные базы данных (РБД) представляют собой организованную систему хранения данных, основанную на таблицах с четко определенными связями между ними. В отличие от NoSQL-решений, таких как MongoDB, реляционные СУБД требуют предварительного определения схемы данных и используют язык SQL (Structured Query Language) для манипуляции информацией.

PostgreSQL — это мощная, открытая объектно-реляционная СУБД, поддерживающая сложные запросы, транзакции, индексы и механизмы репликации. Ее расширяемость и соответствие стандартам SQL делают ее популярным выбором для enterprise-решений.

Установка и настройка PostgreSQL

Перед началом работы необходимо установить PostgreSQL. Доступны варианты локальной установки или использования облачного сервиса (например, ElephantSQL).

1. **Локальная установка**
   - На Windows: загрузить инсталлятор с официального сайта и следовать инструкциям.
   - На Linux (Ubuntu/Debian):

     ```
     sudo apt-get install postgresql postgresql-contrib
     sudo service postgresql start
     ```

После установки сервер PostgreSQL будет доступен на `localhost:5432`. Для подключения используется клиент `psql` или графические инструменты (pgAdmin, DBeaver).

2. **Облачное решение (ElephantSQL)**
   - Регистрация на ElephantSQL.
   - Создание инстанса (бесплатный план `Tiny Turtle`).
   - Получение строки подключения:

```
postgres://<username>:<password>@<host>:5432/<database>
```

Подключение PostgreSQL к Node.js-приложению

Для работы с PostgreSQL в Node.js используются библиотеки `pg` (низкоуровневый драйвер) или ORM (Sequelize, TypeORM). В данном примере рассмотрим `pg` и `Sequelize`.

1. **Инициализация проекта**
   Устанавливаем зависимости:

```
npm init -y
npm install pg sequelize express
```

2. **Подключение через `pg`**
   Создаем файл `server.js`:

```
const { Pool } = require('pg');
const express = require('express');
const app = express();

const pool = new Pool({
    user: 'postgres',
    host: 'localhost',
    database: 'mydatabase',
    password: 'password',
    port: 5432,
});

app.use(express.json());

app.listen(3000, () => {
    console.log('Server is running on http://localhost:3000');
});
```

Для облачного подключения конфигурация будет выглядеть так:

```
const pool = new Pool({
    connectionString: 'postgres://user:password@host:5432/database',
```

```
    ssl: { rejectUnauthorized: false },
});
```

3. **Подключение через Sequelize**
   Sequelize — это ORM (Object-Relational Mapping), который абстрагирует работу с SQL. Инициализация:

```
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize('mydatabase', 'postgres', 'password', {
    host: 'localhost',
    dialect: 'postgres',
});

// Проверка подключения
sequelize.authenticate()
    .then(() => console.log('Connected to PostgreSQL'))
    .catch(err => console.error('Connection error:', err));
```

Определение моделей и связей

В отличие от MongoDB, в PostgreSQL необходимо предварительно создать таблицы. Рассмотрим пример модели "Пользователь" и "Задача" с отношением "один-ко-многим".

1. **Создание таблиц через SQL**

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    user_id INTEGER REFERENCES users(id),
    completed BOOLEAN DEFAULT false
);
```

2. **Определение моделей в Sequelize**

```
const User = sequelize.define('User', {
    name: { type: DataTypes.STRING, allowNull: false },
    email: { type: DataTypes.STRING, unique: true },
});

const Task = sequelize.define('Task', {
    title: { type: DataTypes.STRING },
```

```
    completed: { type: DataTypes.BOOLEAN, defaultValue: false },
});

// Связь 1:N
User.hasMany(Task);
Task.belongsTo(User);

// Синхронизация с БД
sequelize.sync({ force: true }); // Опция `force` пересоздает таблицы
```

CRUD-операции в PostgreSQL

1. Создание записи (Create)

Добавление пользователя через Sequelize:

```
app.post('/users', async (req, res) => {
    try {
        const user = await User.create(req.body);
        res.status(201).send(user);
    } catch (err) {
        res.status(400).send(err.message);
    }
});
```

2. Чтение данных (Read)

Получение пользователей с задачами (JOIN через Sequelize):

```
app.get('/users', async (req, res) => {
    try {
        const users = await User.findAll({ include: Task });
        res.send(users);
    } catch (err) {
        res.status(500).send(err.message);
    }
});
```

3. Обновление данных (Update)

Изменение данных пользователя:

```
app.patch('/users/:id', async (req, res) => {
    try {
        const user = await User.update(req.body, {
            where: { id: req.params.id },
            returning: true, // Для PostgreSQL (возвращает обновленную
```

```
запись)
        });
        res.send(user);
    } catch (err) {
        res.status(400).send(err.message);
    }
});
```

4. Удаление данных (Delete)

Удаление пользователя:

```
app.delete('/users/:id', async (req, res) => {
    try {
        await User.destroy({ where: { id: req.params.id } });
        res.send({ message: 'User deleted' });
    } catch (err) {
        res.status(500).send(err.message);
    }
});
```

Сложные запросы и транзакции

1. **Транзакции**
   Обеспечивают атомарность операций:

   ```
   const result = await sequelize.transaction(async (t) => {
       const user = await User.create({ name: 'John' }, { transaction: t
   });
       await Task.create({ title: 'Learn SQL', userId: user.id }, {
   transaction: t });
       return user;
   });
   ```

2. **Агрегация**
   Пример: подсчет задач по пользователям:

   ```
   const stats = await Task.findAll({
       attributes: [
           'userId',
           [Sequelize.fn('COUNT', Sequelize.col('id')), 'taskCount'],
       ],
       group: ['userId'],
   });
   ```

Интеграция с фронтендом

Пример запроса с фронтенда (Fetch API):

```
fetch('http://localhost:3000/users')
    .then(response => response.json())
    .then(data => console.log(data));
```

Оптимизация

- **Индексы** — ускоряют поиск:

  ```
  CREATE INDEX idx_users_email ON users(email);
  ```

- **Репликация** — повышает отказоустойчивость (настройка в `postgresql.conf`).

Практическое задание

Создайте простое API для управления списком пользователей. Необходимо реализовать серверное приложение с подключением базы данных PostgreSQL для хранения информации о пользователях.

Сущность `Пользователь` должна включать следующий минимальный набор полей (можно добавлять свои):

Поле	Тип данных	Описание
ID	Integer	Уникальный идентификатор пользователя
first_name	Varchar	Имя пользователя
last_name	Varchar	Фамилия пользователя
age	Integer	Возраст пользователя
created_at	Timestamp	Время создания пользователя в unix
updated_at	Timestamp	Время обновления пользователя в unix

Список маршрутов должен включать следующий минимальный перечень энд-поинтов (можно добавлять свои):

Адрес	Метод	Описание
/api/users	POST	Создание нового пользователя
/api/users	GET	Получение списка пользователей
/api/users/:id	GET	Получение конкретного пользователя
/api/users/:id	PATCH	Обновление информации пользователя
/api/users/:id	DELETE	Удаление пользователя

⚠ **Warning**

Обязательно связать логику каждого маршрута с базой данных PostgreSQL!

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.